Laboratory Exercise 08

Topics

1. Lists
2. Tables

Discussion

A. Discuss similarities and differences between lists and tables.
B. Describe the use of indexes in:
    a. strings;
    b. lists;
    c. tables;
C. How do you prevent `an out-of-range error`?

Exercises (in red = suggested ones to complete during the lab)

Part 1 – Processing lists and tables

***Goal**: For each of the following exercises, write a Python program that responds to the above requests. Complete <u>at least two</u> exercises during the lab session, and the remaining ones at home.*

08.1.1 Out-of-range. Write a program that contains an `out-of-range error`. What happens if you run the program? [R6.10]

[SUGGESTED] 08.1.2 Buffer. Write a function `shiftList()` that receives as parameter a list, moves all the elements "forward" by one position (thus increasing their index by one), and puts the element that was originally last, in the first position. For example, the list `1 7 9 3 0 4`, after this operation, becomes the list `4 1 7 9 3 0`. Then write a `main()` that calls this function repeatedly, printing the result every time. [R6.17]

08.1.3 Throwing Dices. Write a program that generates a sequence of `20` random dice rolls, stores them in a list, and displays the generated values, wrapping the longest sequence of identical values between parentheses, with this formatting:

```
1 2 5 5 3 1 2 4 3 (2 2 2 2) 3 6 5 5 6 3 1
```

If there are multiple sequences of identical values of maximum length, use the formatting shown to highlight the first one in order of occurrence. [P6.16]

[SUGGESTED] 08.1.4 Table. Write a Python program to do the following operations with a table of m rows and n columns (dimensions entered from the keyboard):

I. initialize the table with zeros (`0`);
II. fill the entire table with ones (`1`);
III. fill the table by alternating `0` and `1` in a checkerboard pattern;
IV. fill with `0` only the top and bottom rows, leaving the rest of the table unchanged;
V. fill with `1` only leftmost and rightmost columns, leaving the rest of the table unchanged;
VI. Calculate and print the sum of all the elements.

After each operation, print the table. [R6.28]

08.1.5 Merging lists [seen in class]. Write a `merge(a, b)` function that merges the two lists `a` and `b`, alternating an element of the first and an element of the second. If one list is shorter than the other, the elements are alternated as long as possible, then the remaining elements in the longer list are added, in order, at the end of the list. The original lists must not be changed. For example, if the contents of `a` are `1 4 9 16` and the contents of `b` are `9 7 4 9 11`, invoking `merge(a, b)` returns a new list containing the values `1 9 4 7 9 4 16 9 11`. [P6.30]

08.1.6 Merging ordered lists. Write the function `merge_sorted(a, b)` that merges two lists (which are assumed to be already sorted in increasing order), returning a new list that is also sorted in increasing order. Use a "current" index for each list, so you can keep track of the portions that have already been processed. The original lists must not be changed. For example, if the content of `a` is `1 4 9 16` and the content of `b` is `4 7 9 9 11`, the invocation `merge_sorted(a, b)` returns a new list containing the values `1 4 4 7 9 9 9 9 11 16`. Do not use the `sort()` method or the `sorted()` function. [P6.31]

Part 2 – Algorithms that make use of lists and tables

***Goal**: For each of the following exercises, write a Python program that responds to the above requests. Complete <u>at least two</u> exercises during the lab session, and the remaining ones at home.*

[SUGGESTED] 08.2.1 Average. Write the `neighbor_average(values, row, column)` function that, in a `values` table, calculates the average value of an element's neighbors in the eight directions, indexed as shown in the figure below (excluding the element itself). However, if the element is on an edge of the table, the average must be calculated considering only the neighbors that actually belong to the table. For example, if `row` and `column` are both `0`, the element has `3` neighbors. [P6.23]

`[i -1] [j - 1]`	`[i - 1] [j]`	`[i - 1] [j + 1]`
`[i] [j - 1]`	`[i] [j]`	`[i] [j + 1]`
`[i + 1] [j - 1]`	`[i + 1] [j]`	`[i + 1] [j + 1]`

[SUGGESTED] 08.2.2 Magic Squares. An $n \times n$ matrix containing integers $1, 2, 3, \ldots, n^2$ is a "magic square" if the sum of its elements in each row, each column, and the two diagonals has the same value. For example, this is a magic square of size 4:

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

Write a program that takes 16 values as input, arranges them in a 4x4 table in acquisition order, one row at a time from top to bottom, and in each row from left to right, and checks if, after arranging them, they form a magic square. Check two properties:

I. In the acquired data there are all and only the numbers 1, 2, ..., 16.
II. When the numbers are arranged in a table, the sums of the rows, columns, and diagonals are all equal to each other.

[P6.21]

08.2.3 The game of tic-tac-toe. Write a program that plays tic-tac-toe. Tic-tac-toe is played on a 3 × 3 grid. The game is played by two human players who take turns. The first player marks the moves with a circle ('o'), the second with a cross ('x'). The player who has formed a horizontal, vertical or diagonal sequence of 3 signs wins. The program must, at each turn, display the game board, ask the user for the coordinates of the next sign (row and column, numbered from 1 to 3), switch the players after each move and, after each move, check if one of the player has won, or if the game ended in a tie (no more free cells in the grid). [P6.28]

X		O
	X	O
O	O	X

08.2.4 Spring. Write a program that models and simulates the motion of an object of mass m connected to an oscillating spring. When the spring is moved from its equilibrium position by an amount x, Hooke's law states that the restoring force is given by the formula:

$$F = -kx$$

Where k is a constant that depends on the spring. For this simulation, use `k = 10 N/m`. Start with a certain displacement x (e.g., `x = 0.5 m`). Set the initial speed `v = 0`. Calculate acceleration a combining Newton's law ($F = ma$) and Hooke's law, using a mass `m = 1 kg`. Use a small time interval `delta_t = 0.01 s` and, at each step, update the velocity, calculating a variation of $a\Delta t$, and the displacement, calculating a variation of $v\Delta t$. [P6.38]